

UML Diagrams of the Remote File Folder Client Project (a simplified Google Drive analogue)

Yana Utochkina

Stage: Architecture and Behavior Design
September 15, 2026

Abstract

This document presents a set of UML diagrams for the **Remote File Folder Client** project — an application for working with a remote file folder (a simplified analogue of Google Drive) with web and desktop clients, a Go backend, PostgreSQL, and local-folder synchronization built on a server-side version cursor plus client-side local scanning. The diagrams describe the system as it is actually implemented, and are ordered according to the design lifecycle: from requirements analysis and high-level architecture to the detailed design of individual subsystems. All diagrams were built using Mermaid notation.

Contents

1	Requirements Analysis	2
2	Architectural Design (High Level)	3
2.1	Use Case Diagram	3
2.2	Class Diagram (General Structure)	4
2.3	Deployment Diagram (General Architecture)	7
3	Detailed Design	8
3.1	VOPC Class Diagram for the “Synchronize local folder with remote” use case	9
3.2	Object Diagram (Sync-State Snapshot)	11
3.3	Interaction Diagrams (Sequence & Communication Diagrams) . .	13
3.4	State Diagram	21

3.5 Component Diagram	23
3.6 Package Diagram	24
3.7 Activity Diagram	25
4 Conclusions	26

1. Requirements Analysis

The system gives an authenticated user access to a personal remote folder (a “virtual drive”), where they can view the list of files along with their attributes (name, creation date, modification date, uploader, last editor), upload, download, and delete files, as well as synchronize a chosen local directory with the remote one.

Key functional requirements, as actually implemented:

- user registration and authentication on the remote server (username/password, JWT access + refresh tokens);
- viewing file attributes with the ability to show/hide columns (the file name column cannot be hidden);
- viewing the content of specific file types: `.java` - as text, `.png` - as an image;
- sorting files by any column - name, creation date, modification date, uploader, or last editor (ascending / descending);
- filtering files by extension, via a dropdown populated from the extensions actually present among the user’s files;
- uploading either as a single multipart/form-data request, or, for large files, as a chunked session (`initiate` $\rightarrow$ `N  $\times$  chunk` $\rightarrow$ `complete`) that keeps every individual HTTP request under the Cloudflare Tunnel free-tier body-size limit;
- synchronizing the local folder with the remote one: the server reports what changed since the client’s last-seen version (a `download/delete_local` diff only), and the client independently reconciles its own local changes (new/changed/removed files) by uploading, re-uploading, or deleting on the server.

Scope note. REQUIREMENTS.md originally sketched a fuller protocol in which the client also submits a local manifest (name, size, mtime, sha256 per file) so the server could detect genuine bidirectional edit conflicts and offer “keep local / keep remote / keep both” resolution. The shipped `POST /api/sync/diff` endpoint accepts that manifest field for forward compatibility but does not read or compare it - this is a deliberate, documented scope reduction (see `backend/internal/service/sync.go` and `desktop/.../SyncViewModel.swift`) not an oversight. Consequently, if the same file changes on both the local folder and the server between two sync cycles, there is no conflict detection or user prompt: whichever half of the client’s reconcile routine runs first simply wins (last-writer-wins). The diagrams in this report describe the system exactly as built, including this limitation, rather than the originally sketched conflict-resolution design.

Architecturally, the system consists of a React client (web, served by a static-file nginx container), a Swift client (macOS, with a local `FSEvents`-based folder watcher plus a periodic poll), a Go API server (handler / service / repository layers), PostgreSQL for metadata, and a `FileStorage` abstraction currently backed by local disk only (an S3/R2-compatible implementation is a documented future extension point, not yet written), deployed on a home Ubuntu

server and published via Cloudflare Tunnel.

Based on these requirements, the following sections build a use case diagram as well as architectural and detailed diagrams describing the structure and behavior of the system.

2. Architectural Design (High Level)

This stage defines the overall system architecture: who its users are and which scenarios it supports (use case diagram), the general data structure (high-level class diagram), and the physical deployment topology (deployment diagram).

2.1 Use Case Diagram

The diagram (fig. 1) shows a single *User* actor who works with the system through a web or desktop client, and the main use cases corresponding to the functional requirements from section 1, including “*Register*” as an independent entry point. Some of the file operations *include* («include») the “*Authenticate*” use case, since access to the workspace is only possible after a successful login. Viewing file content is *extended* («extend») by two alternative display scenarios (.java / .png). Unlike an earlier version of this diagram, there is no “Resolve sync conflict” use case extending synchronization – as noted in section 1’s scope note, no conflict is ever detected or surfaced to the user in the shipped implementation.

Notation note: Mermaid has no built-in use case diagram type, so the actor, use cases, and system boundary were built using flowchart (actor – rounded rectangle, use cases – “stadium” shape, system boundary – subgraph), with explicit stereotype labels «include»/«extend» on the relations.

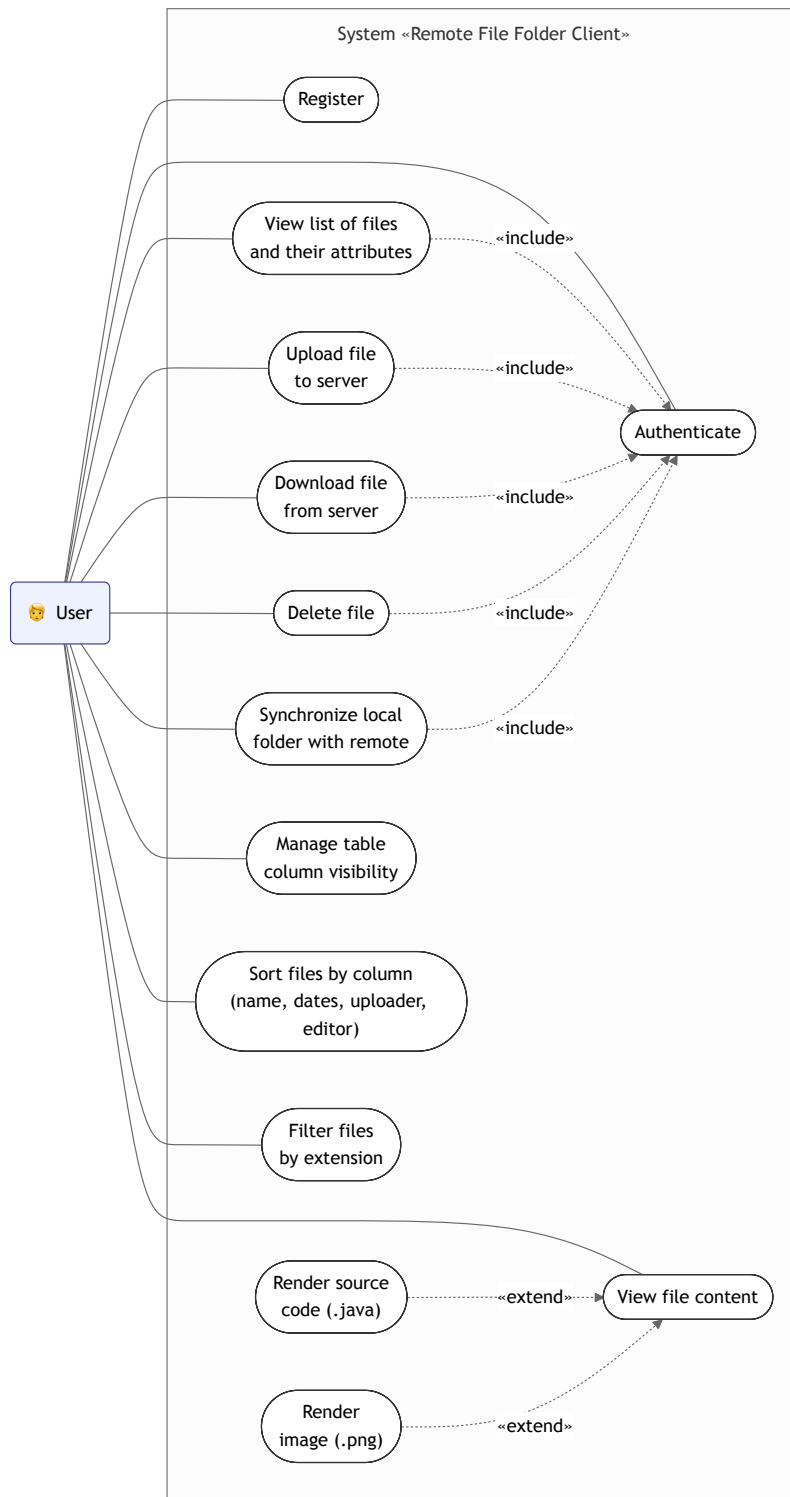


Figure 1: Use case diagram of the Remote File Folder Client system

2.2 Class Diagram (General Structure)

Fig. 2 displays the general data structure and the main relationships between the domain classes of the system, without detailing every method: the User and File entities (matching backend/internal/domain/models.go – note User

carries a username, not an email, and File has no stored sha256), the Deletion tombstone entity that lets the sync endpoint distinguish “deleted” from “never downloaded”, the service layer (AuthService, FileService, SyncService), the data access layer (FileRepo), and the FileStorage abstraction – following the “storage behind an interface” principle laid out in the requirements (sections 5.2, 5.5). Only LocalFileStorage is implemented today; the interface is what would let an S3/R2-compatible backend be added later without touching callers, but no such implementation exists yet, so it is intentionally omitted from the diagram. Likewise, authentication issues stateless JWT access/refresh tokens directly rather than persisting a Session entity – there is no server-side session table or revocation list.

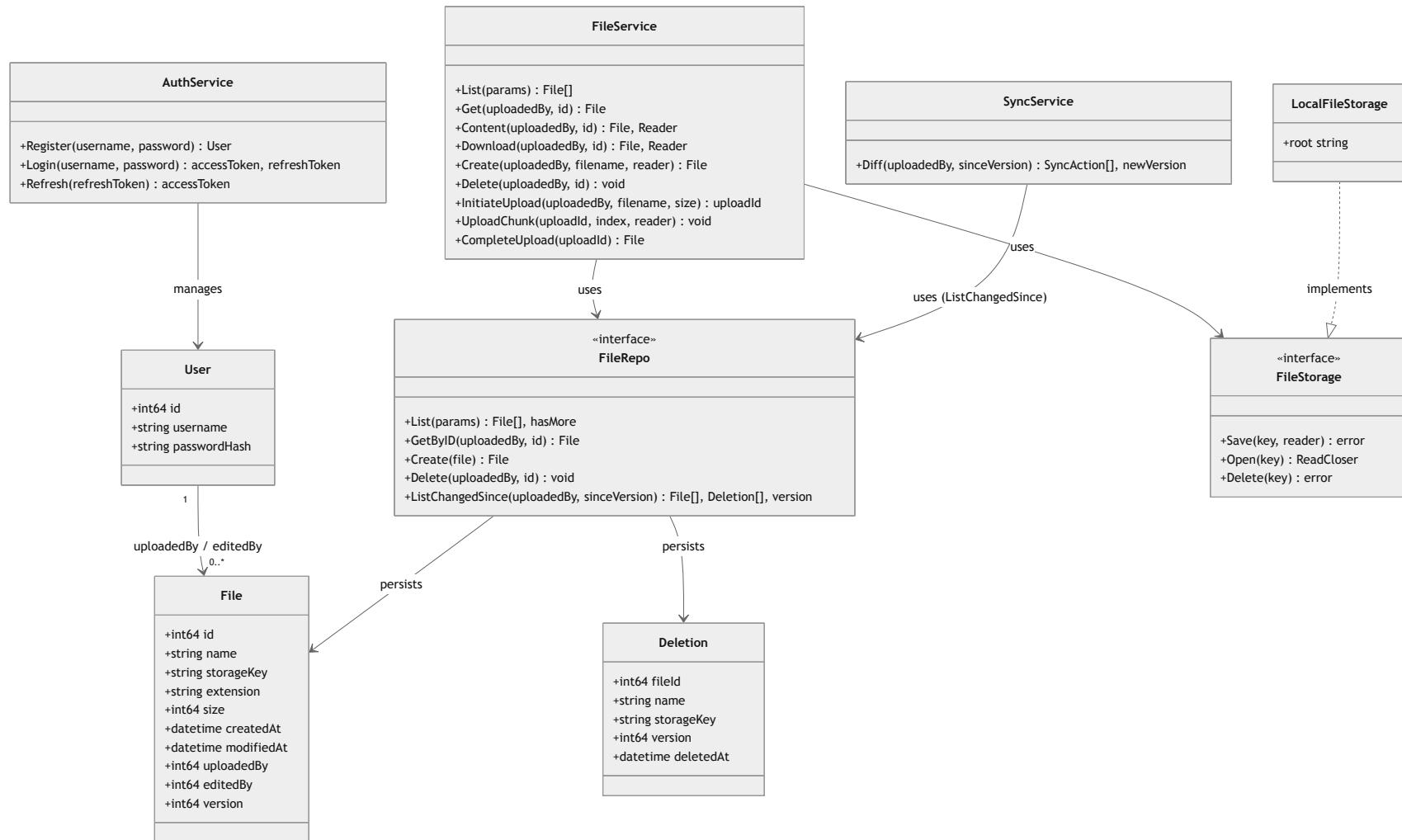


Figure 2: High-level class diagram (domain model)

2.3 Deployment Diagram (General Architecture)

Fig. 3 displays the physical topology of the system: client devices (browser, macOS), the Cloudflare edge (TLS termination, public DNS), and a home Ubuntu server that, via Docker Compose, runs four containers – `cloudflared` (outbound tunnel without opening ports, routing by path), `api` (the Go binary), `web` (nginx serving the built React SPA as static files), and `db` (PostgreSQL) – together with a persistent `pgdata` volume and a bind-mounted `../storage` directory for file content. `cloudflared` routes requests under `/api/*` to `api` and everything else to `web`, so the SPA and the API share one public hostname (see `deploy/docker-compose.yml` and `deploy/cloudflared/config.yml.example`).

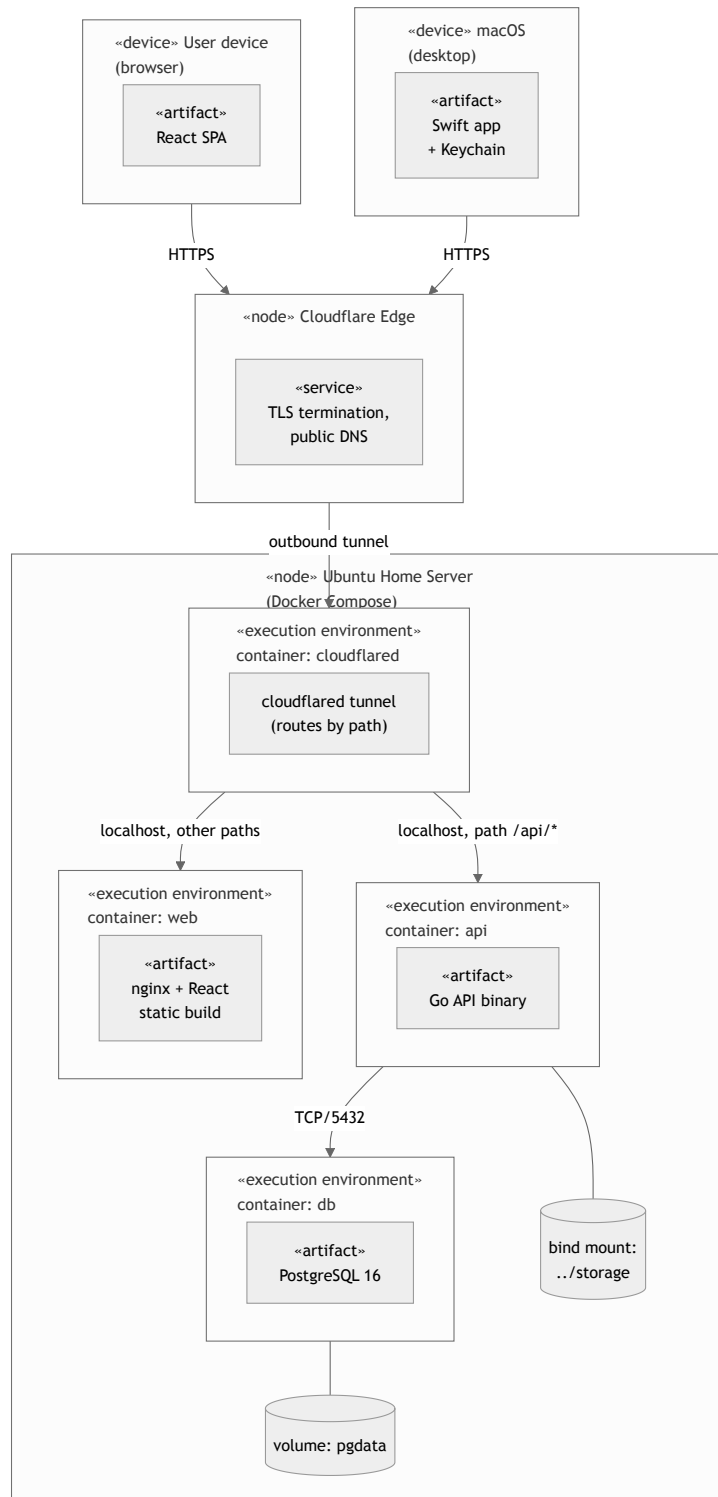


Figure 3: Deployment diagram

3. Detailed Design

Detailed design reveals the internal structure and behavior of individual sub-systems: the classes participating in the implementation of a specific use case (VOPC diagram), a concrete runtime snapshot of objects mid-reconcile (object

diagram), the interaction of objects over time – shown both in a time-ordered (sequence) and a structural (communication) notation –, the behavior of a file depending on its synchronization state (state diagram), the structure of software components (component diagram), the modular organization of the code (package diagram), and the business-process logic of synchronization (activity diagram).

3.1 VOPC Class Diagram for the “Synchronize local folder with remote” use case

According to the assignment variant, the use case chosen for the VOPC diagram (*View Of Participating Classes*) is “**Synchronize local folder with remote**” – the most complex scenario in the system, now shown with the classes that actually implement it in `desktop/Sources/FileSpaceDesktop/` and `backend/internal/handler,service/sync.go`. Classes are distributed according to standard analysis stereotypes:

- «boundary» – user- and OS-facing elements: `SyncPanelView` (choose folder / start / stop), `DirectoryWatcher` (the `FSEvents` wrapper that raises local-change events);
- «control» – control logic: `SyncViewModel` (client-side reconcile loop), `FolderScanner` (computes the local manifest), `SyncHandler/SyncService` (server, one-way diff computation);
- «entity» – domain data: `SyncState` (persisted cursor and per-file known entries), `SyncedFileEntry`, `File`, `Deletion`.

There is deliberately no `ConflictResolver` or `ConflictDialogUI` in this diagram: as explained in section 1’s scope note, the server never reports a conflict action, so no client-side class exists to resolve one.

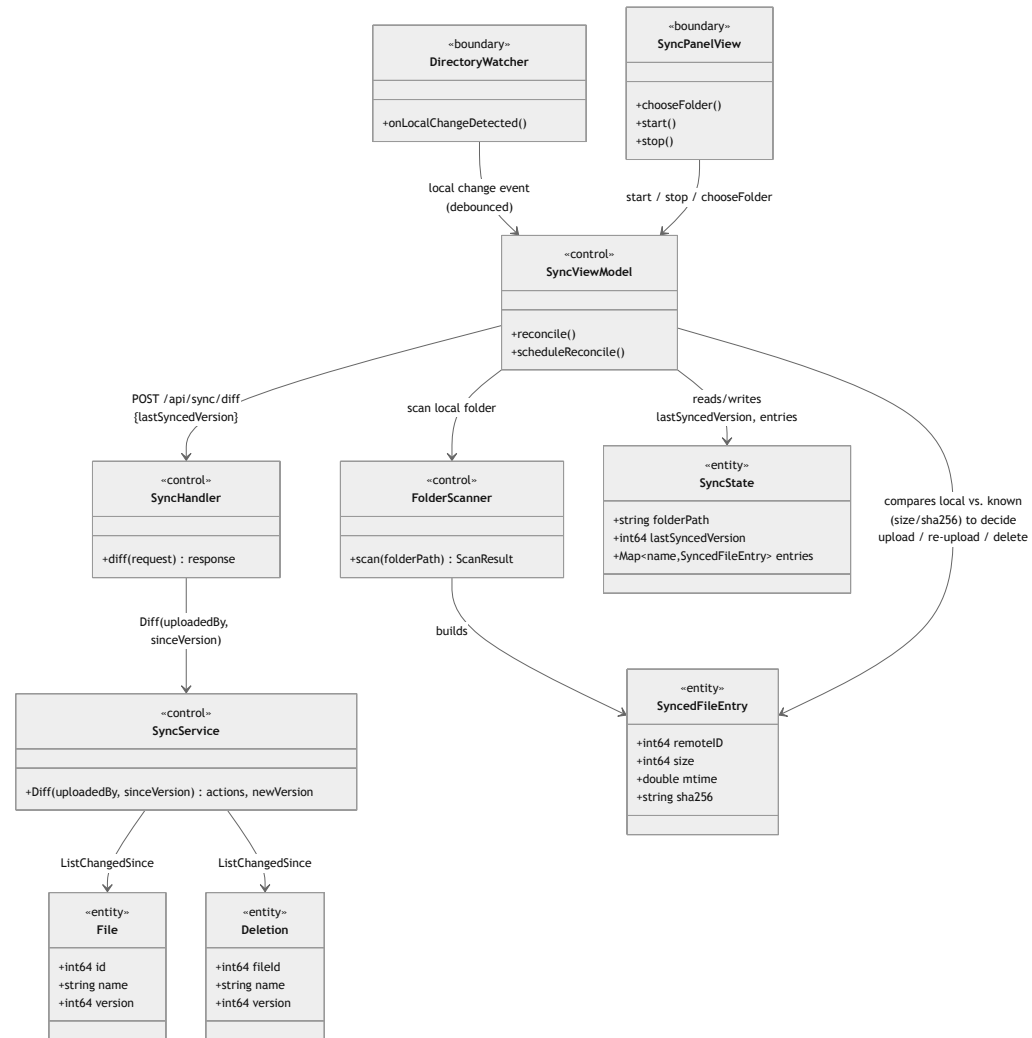


Figure 4: VOPC class diagram for the “Synchronize local folder with remote” use case

3.2 Object Diagram (Sync-State Snapshot)

Unlike class diagrams, which describe the structure of types, an object diagram (fig. 5) captures a *concrete runtime snapshot* of instances - one moment inside `SyncViewModel.reconcile()` at which a locally modified file is about to be re-uploaded. The persisted state `1 : SyncState` holds `lastSyncedVersion = 42` and a known entry for `Notes.java` (`remoteID = 8241`, the size/sha256 recorded the last time this client synced it). `FolderScanner` produces a fresh `local : LocalFileEntry` for the same file whose size/sha256 no longer match the known entry, so `SyncViewModel` decides to delete remote file 8241 and re-upload it as a new row (there is no update-in-place endpoint). As the note on the diagram makes explicit, this is a *race*, not a detected conflict: if `remoteFile` had also changed server-side since version 42, the very next `POST /api/sync/diff` call would independently report a download action for it, and neither side would ever learn the other had changed too.

Notation note: Mermaid has no built-in object diagram type, so the instances were built using flowchart: each object's header is labeled per the UML convention as instance_name : Class, with concrete attribute values listed below a separator.

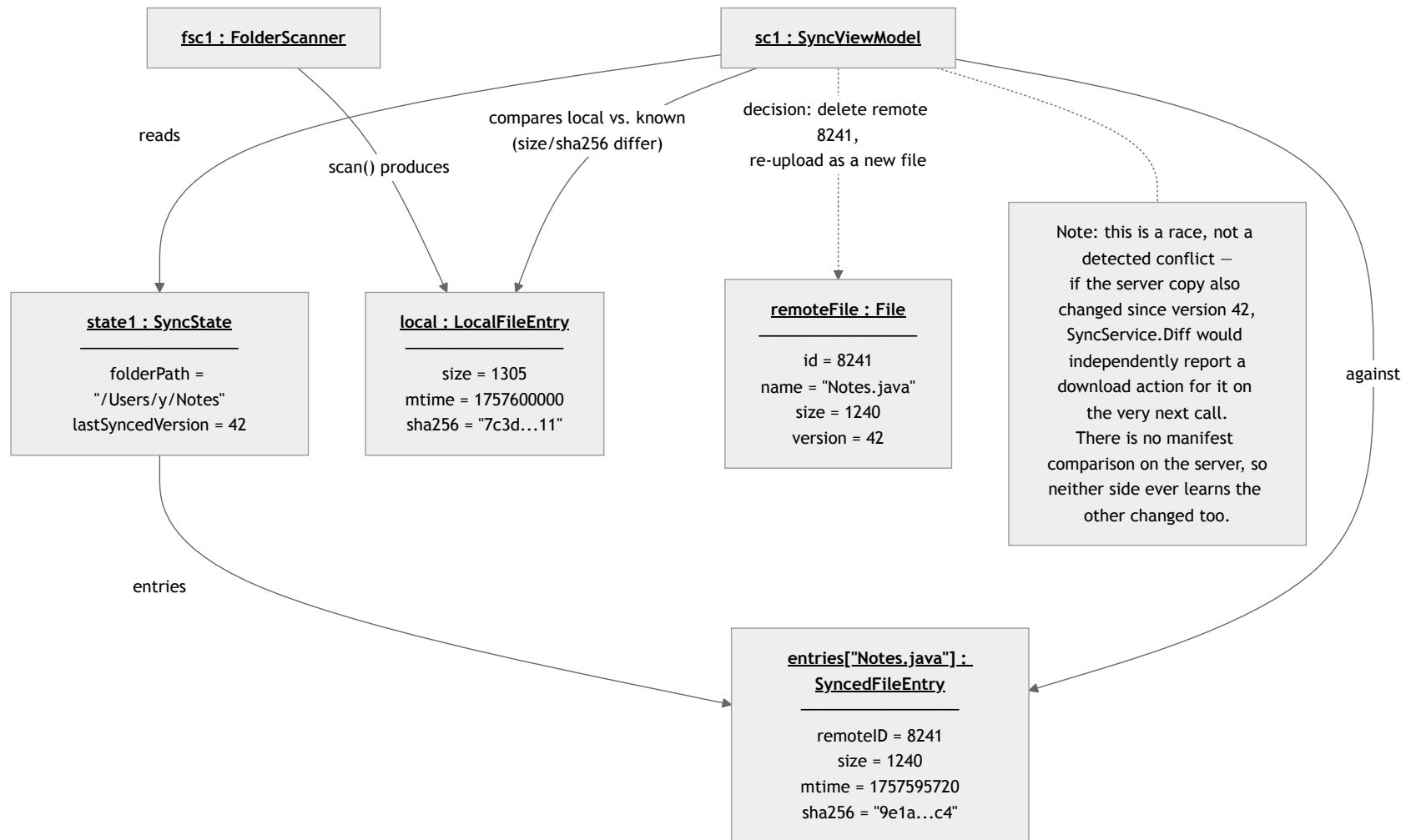


Figure 5: Object diagram: sync-state snapshot just before a locally modified file is re-uploaded

3.3 Interaction Diagrams (Sequence & Communication Diagrams)

Three sequence diagrams are presented, illustrating key interaction scenarios between the client, the Go API server, and the database, together with a communication diagram as an alternative representation of one of these scenarios.

Authentication

Fig. 6 shows the sequence of verifying user credentials (by username, not email) and issuing a token pair (access/refresh) as two independently signed, stateless JWTs – there is no server-side session row to create or look up.

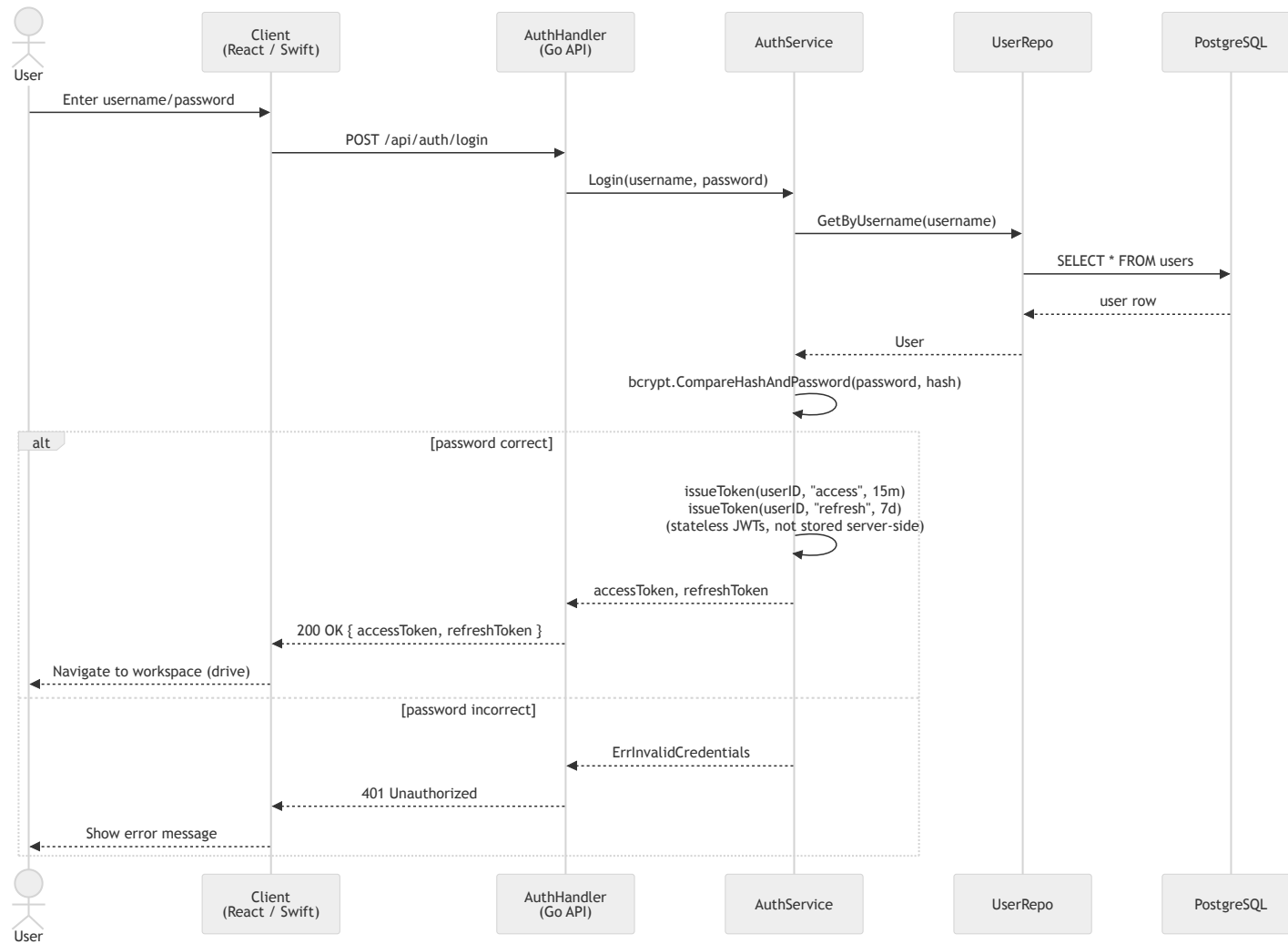


Figure 6: Sequence diagram: user authentication

File Upload

Fig. 7 illustrates the path of a request from the client through the handler layer to the service layer, which first writes the file content to storage (`FileStorage`), and after the write is confirmed, writes the file metadata to PostgreSQL via `FileRepo`, retrying with an auto-generated “name (n).ext” if the name collides with one the same user already has. A note on the diagram points to the chunked-upload variant of the same path, used for large files.

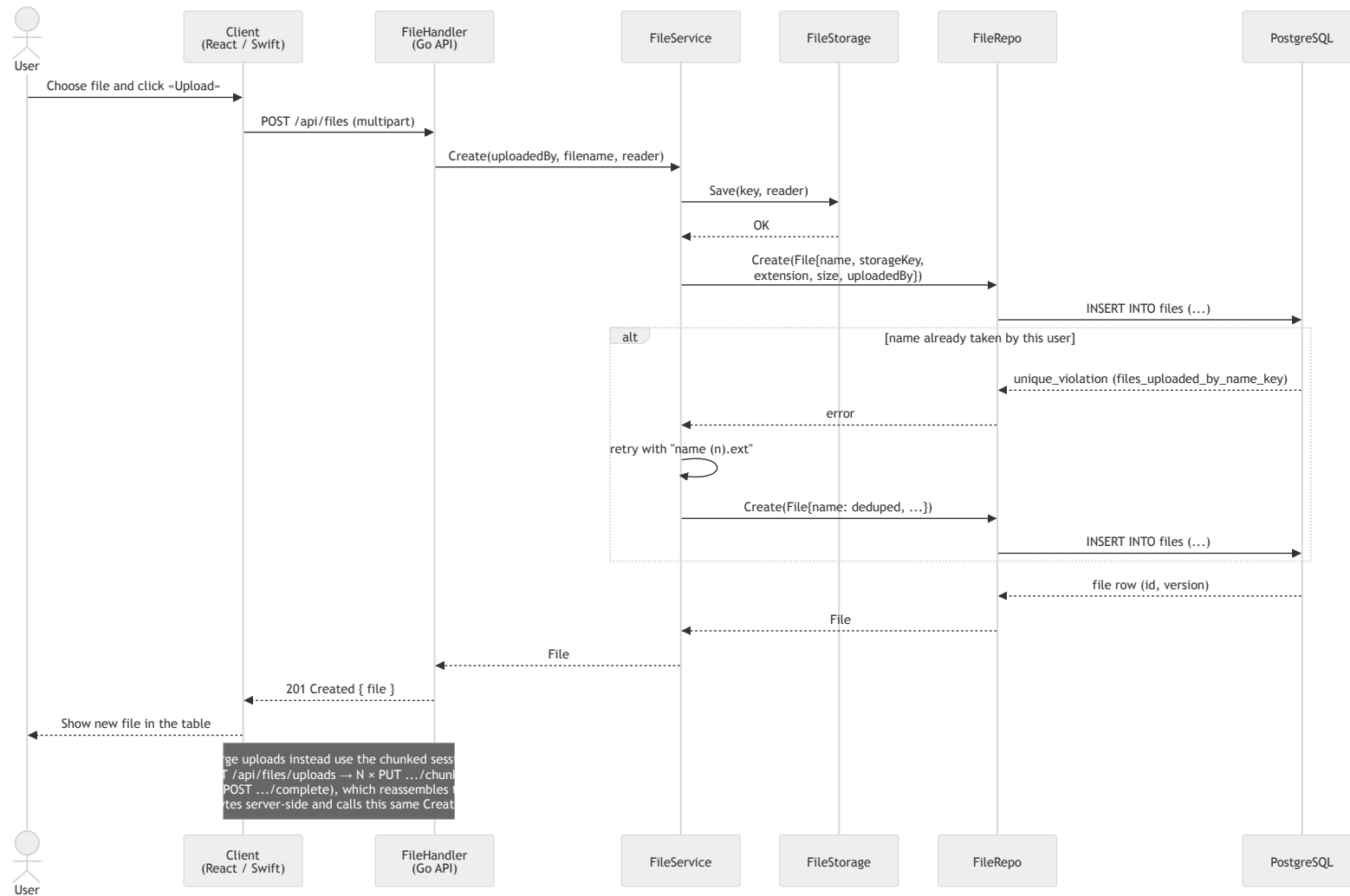


Figure 7: Sequence diagram: uploading a file to the server

Synchronization

Fig. 8 details the actual `POST /api/sync/diff` round trip: the client sends only `lastSyncedVersion` (no manifest is read server-side), the server returns `download/delete_local` actions plus a `newVersion`, the client applies those, and then - independently, with no server involvement in deciding - scans its local folder and uploads, re-uploads, or deletes files based on its own locally persisted `SyncState`. As the note on the diagram states, there is no conflict branch: a same-file change on both sides between two cycles is resolved by whichever half of this routine happens to run first.

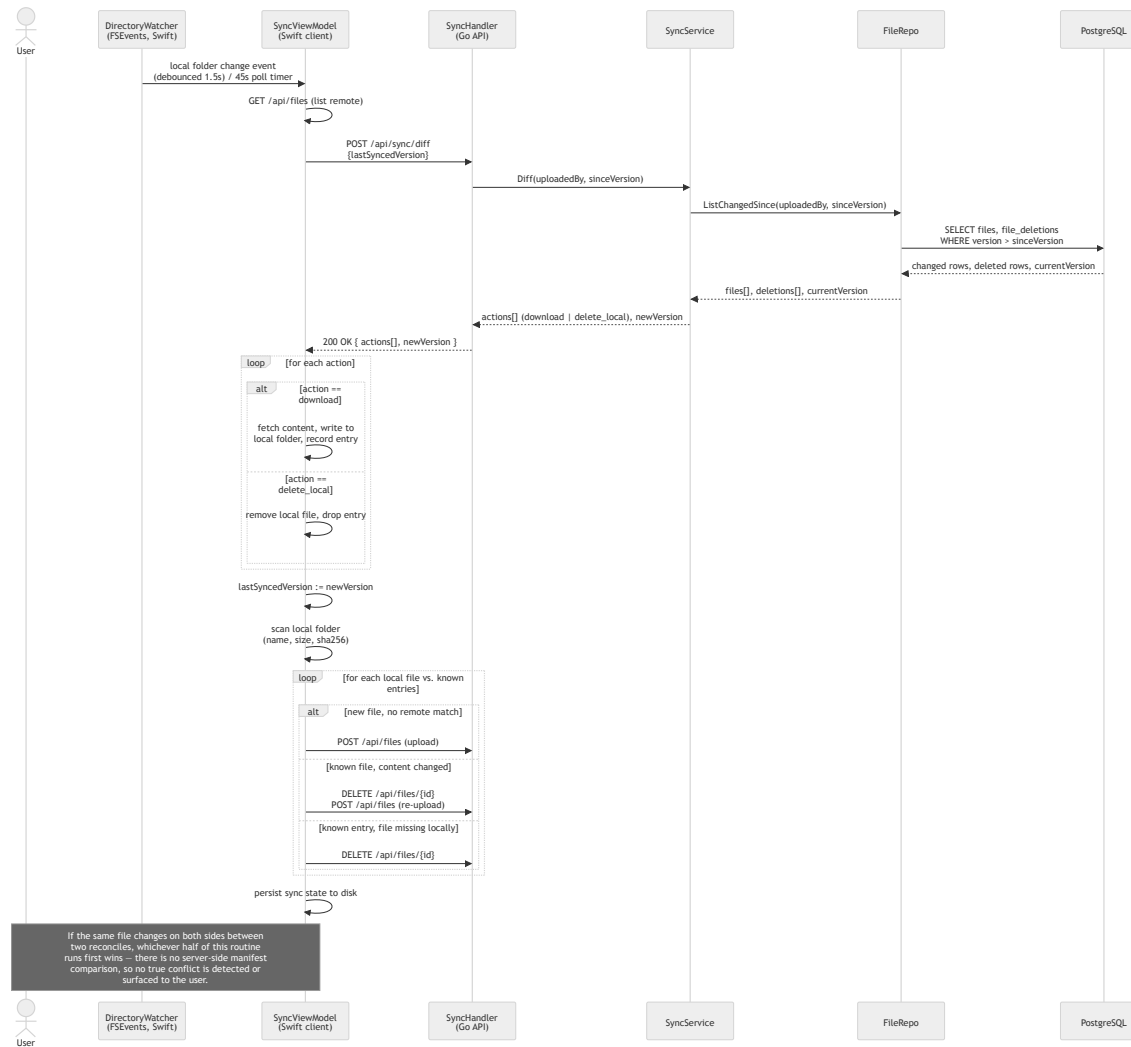


Figure 8: Sequence diagram: synchronization (server diff + client-side reconcile)

The Same Scenario as a Communication Diagram

Fig. 9 depicts **the same** synchronization scenario as fig. 8, but in communication-diagram notation: the emphasis shifts from the time axis to the static links between objects, and message ordering is conveyed through numbering. Both diagrams are consistent with each other – they use the same objects and messages – and together demonstrate the two standard ways of depicting one interaction.

Notation note: Mermaid has no built-in communication diagram type, so it was built using flowchart: objects are graph nodes, and the numbered edge labels correspond to messages.

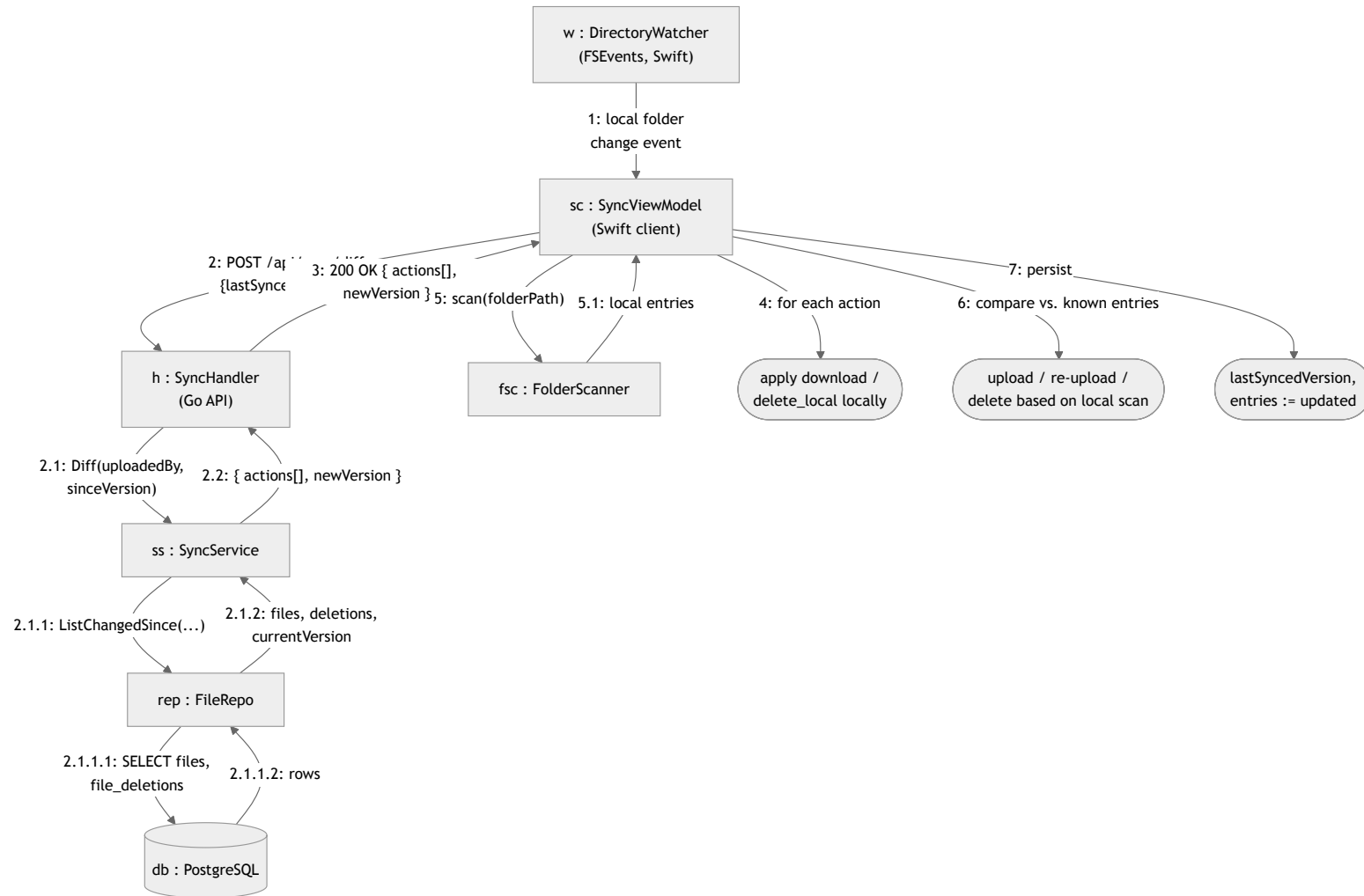


Figure 9: Communication diagram: synchronization (server diff + client-side reconcile)

3.4 State Diagram

Fig. 10 describes the behavior of a file depending on its synchronization state, from the client's point of view: from local/remote creation, through transfer (Uploading/Downloading), to Synced, possible local or remote changes (which loop back into a re-upload or a download), and deletion on either side (a remote deletion arrives as a `delete_local` diff action; a local deletion is detected by the file's known entry going missing from a folder scan). As the note on the diagram states, there is deliberately no Conflict state: a simultaneous change on both sides is a race resolved by last-writer-wins, not a modeled transition.

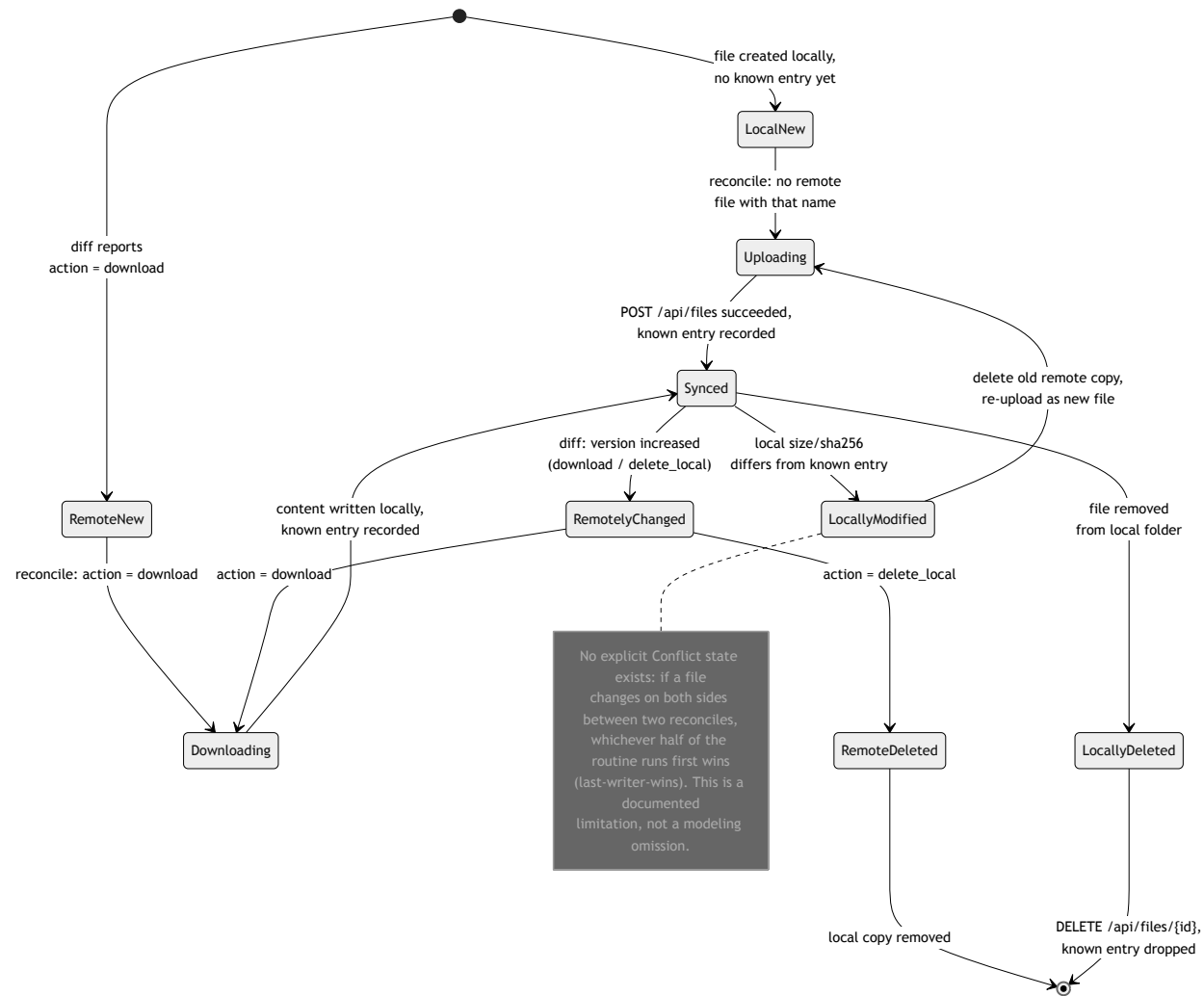


Figure 10: State diagram of a file during synchronization

3.5 Component Diagram

Fig. 11 details the software components of the system and their dependencies: the client applications, the cloudflared tunnel (which routes by path to either the api or the web container), the web nginx component serving the built React SPA, and, inside the Go API server, the Handler → Service → Repository layers, together with the implementation-independent FileStorage interface, whose only current implementation is local-disk storage (sections 5.2, 5.5 of the requirements).

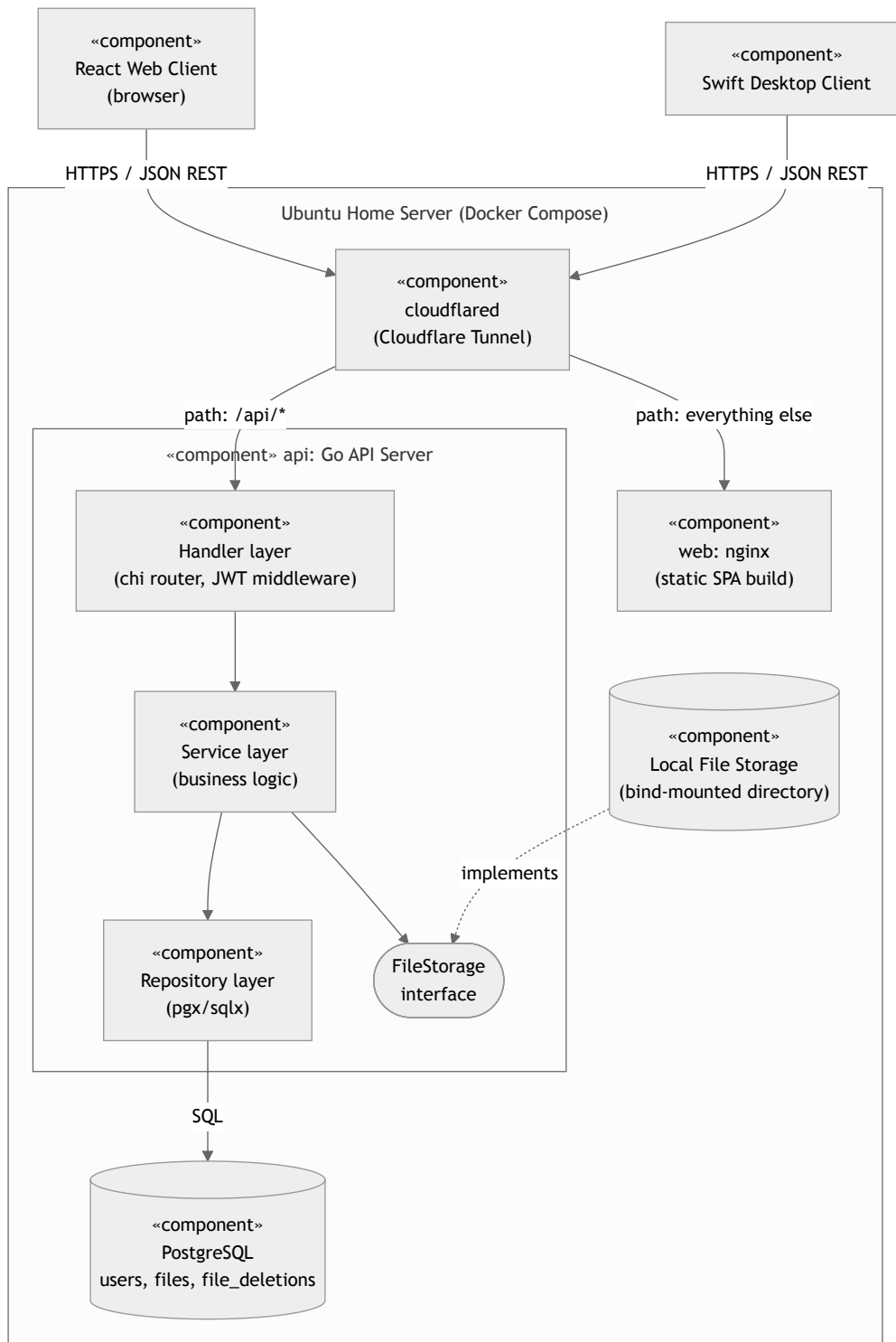


Figure 11: Component diagram of the system

3.6 Package Diagram

Where the component diagram (fig. 11) shows runtime components and the interfaces between them, the package diagram (fig. 12) shows the *static* organization of the code into modules, matching the actual repository layout

(backend, frontend, desktop, migrations): inside the Go module, a layered package structure handler → service → repo/storage, built on shared support packages (domain, db, config, authctx), matching the “no direct filesystem calls from handlers” convention and the principle of accessing storage only through the FileStorage interface. The repo package additionally depends on the schema defined in migrations. The desktop package groups SwiftUI views separately from the sync engine (DirectoryWatcher, SyncViewModel, and Manifest.swift’s SyncState/FolderScanner) and the shared APIClient/KeychainStore and the frontend package now also depends on TanStack Virtual for windowing the file table alongside TanStack Query and Table.

Notation note: Mermaid has no built-in package diagram type, so packages are shown as nested subgraph blocks (with a «package» stereotype in the title), and dependencies as dashed arrows labeled with the dependency action.

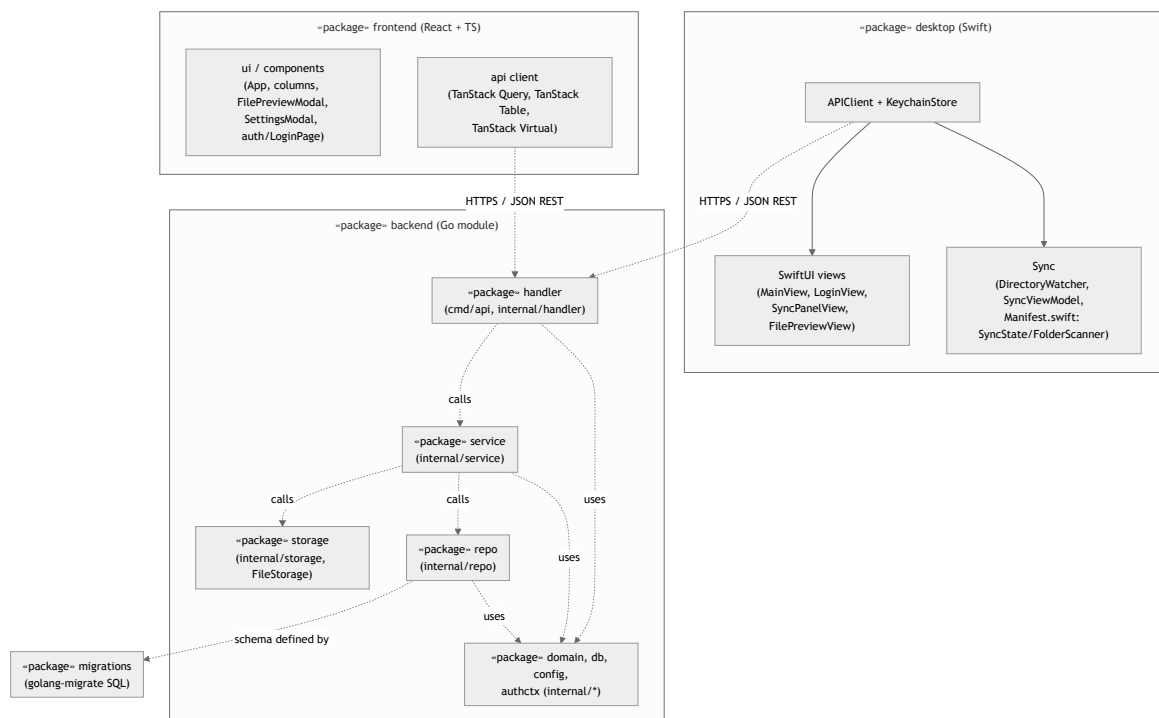


Figure 12: Package diagram: code organization by module

3.7 Activity Diagram

Fig. 13 details the business logic of a single reconcile cycle, triggered either by an FSEvents change notification or a 45-second poll: listing remote files and requesting the server-authoritative diff since lastSyncedVersion, applying each download/delete_local action and saving the new version, then scanning the local folder and, independently of the server, uploading new files, deleting-and-re-uploading changed files, and deleting server-side copies of files removed locally, before persisting the updated sync state to disk.

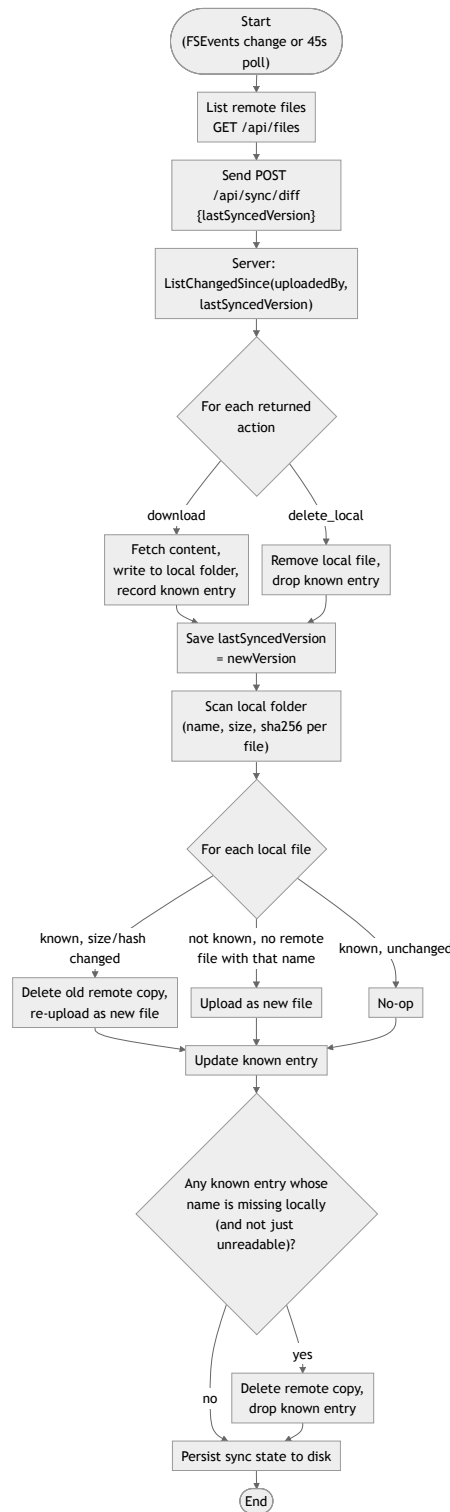


Figure 13: Activity diagram: synchronization cycle

4. Conclusions

The developed set of diagrams consistently reveals the Remote File Folder Client system from business requirements to detailed structure and behavior, as it is actually implemented rather than as originally sketched: the use case

diagram captures the user's functional scenarios, including registration; the general class diagram and the (now four-container) deployment diagram define the architectural framework; the VOPC diagram, the object diagram, the sequence and communication diagrams, and the state, component, package, and activity diagrams detail the implementation of the most complex scenario - synchronizing files between the local and remote folder. That scenario turned out simpler than originally scoped: the server exposes a one-way, version-cursor diff (download/delete_local only), and the desktop client reconciles its own local changes independently, with no manifest-based conflict detection or resolution UI - a deliberate, documented trade-off rather than a missing feature. In particular, the object diagram and the communication diagram complement the VOPC diagram and the sequence diagrams, respectively, with a concrete state snapshot (a re-upload race, not a resolved conflict) and an alternative (structural rather than time-based) notation for the same scenario, while the package diagram captures the static modular organization of the code, consistent with the actual repository layout. The constructed diagrams are mutually consistent (shared classes, entities, and actions) and serve as an accurate foundation for the further development and testing of the system.